

Docker gyakorlás

Statikus weboldal futtatása Nginx konténerből

Mount és Volume különbsége érthetően

Egyszerű HTML oldal futtatása nginx-ben

Bind mount + volume kipróbálása → Különbségek megértése

Megoldás

1.1. Projekt mappa létrehozása

```
mkdir nginx-static  
cd nginx-static
```

1.2. Készíts egy egyszerű HTML oldalt

```
<!DOCTYPE html>  
<html>  
<head>  
  <title>Docker Nginx teszt</title>  
</head>  
<body>  
  <h1>Hello Docker! - Krisztián teszt oldala</h1>  
</body>  
</html>
```

1.3. Futtatás bind mount-tal

```
docker run -d -p 8080:80 -v $(pwd):/usr/share/nginx/html --name my-nginx nginx
```

ellenőrzés

<http://localhost:8080>

Bind mount vs Volume – röviden és érthetően

	Bind mount	Volume
Hol tárolódik?	Bárhol a host fájlrendszerén	Docker által kezelt terület pl. <code>/var/lib/docker/volumes/</code>
Mire jó?	Fejlesztéshez (élő frissítés)	Tartós adat (DB, config) megőrzésére
Látható a hoston?	Igen	Kevésbé (Docker kezeli)
Biztonság	Kevésbé biztonságos	Stabil, Docker kezeli
Előny	Rugalmas	Hordozható

1.4. Futtatás volume-mal

```
docker volume create nginx_data  
``
```

Majd futtasd:

```
docker run -d -p 8081:80 -v nginx_data:/usr/share/nginx/html --name nginx-volume nginx
```

A volume-n nem lesz ott a te HTML-ed, mert az nginx saját alapértelmezett tartalmát tölti be.

Egyszerű log vizualizáció: Grafana + Loki

Docker Compose segítségével

2.1. Projekt mappa

```
mkdir grafana-loki  
cd grafana-loki
```

2.2. **docker-compose.yml** létrehozása

```
version: "3.9"
```

```
services:
```

```
  loki:
```

```
    image: grafana/loki:2.9.0
```

```
    ports:
```

```
      - "3100:3100"
```

```
    command: -config.file=/etc/loki/local-config.yaml
```

```
promtail:
  image: grafana/promtail:2.9.0
  volumes:
    - /var/log:/var/log
    - ./promtail-config.yaml:/etc/promtail/promtail-config.yaml
  command: -config.file=/etc/promtail/promtail-config.yaml
```

```
grafana:
  image: grafana/grafana:latest
  ports:
    - "3000:3000"
  depends_on:
    - loki
```

2.3. Promtail konfiguráció

promtail-config.yaml:

```
server:
  http_listen_port: 9080

positions:
  filename: /tmp/positions.yaml

clients:
  - url: http://loki:3100/loki/api/v1/push

scrape_configs:
  - job_name: system
    static_configs:
      - targets:
          - localhost
    labels:
      job: varlogs
      __path__: /var/log/*log
```

2.4. Stack indítása

docker compose up -d

2.5. Grafana megnyitása

<http://localhost:3000>

Default login:

felhasználó: **admin**

jelszó: **admin**

2.6. Loki hozzáadása adatforrásként

Grafana → Configuration → Data sources → Add → Loki

<http://localhost:3100>

Portainer telepítése – Docker kezelése GUI-ból

Portainer egy nagyon hasznos Docker/WebUI, tökéletes kezdőknek és rendszergazdáknak.

3.1. Docker volume létrehozása Portainer számára

```
docker volume create portainer_data
```

3.2. Portainer telepítése

```
docker run -d |  
-p 9443:9443 |  
-p 8000:8000 |  
--name portainer |  
--restart=always |  
-v /var/run/docker.sock:/var/run/docker.sock |  
-v portainer_data:/data |  
portainer/portainer-ce:latest
```

Megnyitás:

<https://localhost:9443>

Első belépéskor létrehozol egy admin felhasználót.

3.3. Mit tudsz gyakorolni Portainerrel?

- konténer indítás/leállítás
- képek törlése
- volume-ok kezelése
- saját stack indítása (docker-compose fájlal)
- logok megtekintése
- hálózatok konfigurálása